

Cursor introduction, context systems, and scientific workflows

Cursor is a code editor that lets you delegate bounded software tasks to AI agents while inspecting, constraining, and verifying their work.

Cursor is a code editor that lets you delegate bounded software tasks to AI agents while inspecting, constraining, and verifying their work.

Treat it as a lab assistant, not an oracle.

What Cursor changes

- It can edit multiple files across a codebase.

What Cursor changes

- It can edit multiple files across a codebase.
- It can plan before editing.

What Cursor changes

- It can edit multiple files across a codebase.
- It can plan before editing.
- It can use project context, rules, skills, and subagents.

What Cursor changes

- It can edit multiple files across a codebase.
- It can plan before editing.
- It can use project context, rules, skills, and subagents.
- It can run commands and inspect failures.

What Cursor changes

- It can edit multiple files across a codebase.
- It can plan before editing.
- It can use project context, rules, skills, and subagents.
- It can run commands and inspect failures.
- It can make large mistakes quickly.

What Cursor changes

- It can edit multiple files across a codebase.
- It can plan before editing.
- It can use project context, rules, skills, and subagents.
- It can run commands and inspect failures.
- It can make large mistakes quickly.

What Cursor changes

- It can edit multiple files across a codebase.
- It can plan before editing.
- It can use project context, rules, skills, and subagents.
- It can run commands and inspect failures.
- It can make large mistakes quickly.

The workflow matters more than the tool.

- Ask: read-only exploration and Q&A.

- **Ask**: read-only exploration and Q&A.
- **Plan**: codebase research + clarifying questions + plan draft.

- **Ask**: read-only exploration and Q&A.
- **Plan**: codebase research + clarifying questions + plan draft.
- **Agent**: implementation with tools.

- **Ask**: read-only exploration and Q&A.
- **Plan**: codebase research + clarifying questions + plan draft.
- **Agent**: implementation with tools.
- Good workflow: switch by task phase.

- Agent inspects the codebase before editing.

- Agent inspects the codebase before editing.
- Agent asks clarifying questions before coding.

- Agent inspects the codebase before editing.
- Agent asks clarifying questions before coding.
- You review and edit the generated plan.

- Agent inspects the codebase before editing.
- Agent asks clarifying questions before coding.
- You review and edit the generated plan.
- Build starts only after your approval.

- Agent inspects the codebase before editing.
- Agent asks clarifying questions before coding.
- You review and edit the generated plan.
- Build starts only after your approval.
- The approved plan governs the implementation context.

- Agent inspects the codebase before editing.
- Agent asks clarifying questions before coding.
- You review and edit the generated plan.
- Build starts only after your approval.
- The approved plan governs the implementation context.

- Agent inspects the codebase before editing.
- Agent asks clarifying questions before coding.
- You review and edit the generated plan.
- Build starts only after your approval.
- The approved plan governs the implementation context.

A plan is a contract, not a guarantee.

Delegation is safe only when the task has boundaries.

Delegation is safe only when the task has boundaries.

- Define the scientific goal.

Delegation is safe only when the task has boundaries.

- Define the scientific goal.
- Define the files or modules in scope.

Delegation is safe only when the task has boundaries.

- Define the scientific goal.
- Define the files or modules in scope.
- Define non-goals.

Delegation is safe only when the task has boundaries.

- Define the scientific goal.
- Define the files or modules in scope.
- Define non-goals.
- Define verification commands before implementation.

Delegation is safe only when the task has boundaries.

- Define the scientific goal.
- Define the files or modules in scope.
- Define non-goals.
- Define verification commands before implementation.
- Stop the agent when it leaves the contract.

Why planning improves reliability

- Forces precise goals and constraints.

Why planning improves reliability

- Forces precise goals and constraints.
- Allows context control.

Why planning improves reliability

- Forces precise goals and constraints.
- Allows context control.
- Surfaces hidden assumptions early.

Why planning improves reliability

- Forces precise goals and constraints.
- Allows context control.
- Surfaces hidden assumptions early.
- Cuts random retries during implementation.

Why planning improves reliability

- Forces precise goals and constraints.
- Allows context control.
- Surfaces hidden assumptions early.
- Cuts random retries during implementation.
- Makes results easier to reproduce and review.

What a detailed plan means

- Exact files and symbols to change.

What a detailed plan means

- Exact files and symbols to change.
- Ordered steps with dependencies.

What a detailed plan means

- Exact files and symbols to change.
- Ordered steps with dependencies.
- Verification command per step.

What a detailed plan means

- Exact files and symbols to change.
- Ordered steps with dependencies.
- Verification command per step.
- Rollback or fallback for risky changes.

A good prompt specifies:

A good prompt specifies:

- goal: what should change

A good prompt specifies:

- goal: what should change
- scope: what files or modules are in bounds

A good prompt specifies:

- goal: what should change
- scope: what files or modules are in bounds
- non-goals: what must not change

A good prompt specifies:

- goal: what should change
- scope: what files or modules are in bounds
- non-goals: what must not change
- assumptions: scientific and mathematical constraints

A good prompt specifies:

- goal: what should change
- scope: what files or modules are in bounds
- non-goals: what must not change
- assumptions: scientific and mathematical constraints
- verification: exact commands to run

A good prompt specifies:

- goal: what should change
- scope: what files or modules are in bounds
- non-goals: what must not change
- assumptions: scientific and mathematical constraints
- verification: exact commands to run
- stopping rule: when to report back instead of improvising

Bad prompt vs good prompt

Bad:

Bad prompt vs good prompt

Bad:

Make an ABM about polarization.

Bad prompt vs good prompt

Bad:

Make an ABM about polarization.

Better:

Bad prompt vs good prompt

Bad:

Make an ABM about polarization.

Better:

Create a small Python agent-based model with 100 agents on a ring network. Each agent has one belief in $[-1, 1]$. Implement the update rule described in README.md. Do not add a GUI. Add tests that check belief bounds, deterministic seeds, and output file creation. Run unittest and run_sim.py.

Bad prompt vs good prompt

Bad:

Make an ABM about polarization.

Better:

Create a small Python agent-based model with 100 agents on a ring network. Each agent has one belief in $[-1, 1]$. Implement the update rule described in README.md. Do not add a GUI. Add tests that check belief bounds, deterministic seeds, and output file creation. Run unittest and run_sim.py.

Better still: ESL prompt

- Clear non-goals (what not to touch).

- Clear non-goals (what not to touch).
- Data or math assumptions stated explicitly (and correct).

- Clear non-goals (what not to touch).
- Data or math assumptions stated explicitly (and correct).
- Acceptance tests named in advance (and correct).

- Clear non-goals (what not to touch).
- Data or math assumptions stated explicitly (and correct).
- Acceptance tests named in advance (and correct).
- Risks and unknowns listed, not hidden.

Plan quality checklist

- Clear non-goals (what not to touch).
- Data or math assumptions stated explicitly (and correct).
- Acceptance tests named in advance (and correct).
- Risks and unknowns listed, not hidden.
- Follow TDD principles: test first, then code.

- Clear non-goals (what not to touch).
- Data or math assumptions stated explicitly (and correct).
- Acceptance tests named in advance (and correct).
- Risks and unknowns listed, not hidden.
- Follow TDD principles: test first, then code.
- Would a human reviewer understand the code without the chat transcript?

- Rules: persistent constraints, often project-level.

- Rules: persistent constraints, often project-level.
- Skills: reusable procedures, often task-level or user-wide.

- Rules: persistent constraints, often project-level.
- Skills: reusable procedures, often task-level or user-wide.
- Subagents: isolated specialists (with own context).

- Rules: persistent constraints, often project-level.
- Skills: reusable procedures, often task-level or user-wide.
- Subagents: isolated specialists (with own context).
- Hooks/commands: workflow automation (e.g., hooks to run tests after each edit).

More context is not always better.

More context is not always better.

- Irrelevant files distract the model.

More context is not always better.

- Irrelevant files distract the model.
- Stale instructions conflict with current goals.

More context is not always better.

- Irrelevant files distract the model.
- Stale instructions conflict with current goals.
- Long chats hide the active contract.

More context is not always better.

- Irrelevant files distract the model.
- Stale instructions conflict with current goals.
- Long chats hide the active contract.
- Skills and subagents should narrow context, not bloat it.

- Rule: never change the research question without asking.

- Rule: never change the research question without asking.
- Rule: never edit protected test files.

Rules vs skills vs subagents

- Rule: never change the research question without asking.
- Rule: never edit protected test files.
- Skill: run a simulation verification ritual.

- Rule: never change the research question without asking.
- Rule: never edit protected test files.
- Skill: run a simulation verification ritual.
- Skill: perform a safe refactor.

Rules vs skills vs subagents

- Rule: never change the research question without asking.
- Rule: never edit protected test files.
- Skill: run a simulation verification ritual.
- Skill: perform a safe refactor.
- Subagent: inspect the codebase for untested assumptions.

Rules vs skills vs subagents

- Rule: never change the research question without asking.
- Rule: never edit protected test files.
- Skill: run a simulation verification ritual.
- Skill: perform a safe refactor.
- Subagent: inspect the codebase for untested assumptions.
- Subagent: research documentation in parallel.

Rules vs skills vs subagents

- Rule: never change the research question without asking.
- Rule: never edit protected test files.
- Skill: run a simulation verification ritual.
- Skill: perform a safe refactor.
- Subagent: inspect the codebase for untested assumptions.
- Subagent: research documentation in parallel.

Rules vs skills vs subagents

- Rule: never change the research question without asking.
- Rule: never edit protected test files.
- Skill: run a simulation verification ritual.
- Skill: perform a safe refactor.
- Subagent: inspect the codebase for untested assumptions.
- Subagent: research documentation in parallel.

Combine them only when they reduce confusion.

- Project: `.cursor/skills/<skill>/SKILL.md`

- Project: `.cursor/skills/<skill>/SKILL.md`
- User-wide: `~/.cursor/skills/`

- Project: `.cursor/skills/<skill>/SKILL.md`
- User-wide: `~/.cursor/skills/`
- Frontmatter: `name, description` (when Agent should apply it)

- Project: `.cursor/skills/<skill>/SKILL.md`
- User-wide: `~/.cursor/skills/`
- Frontmatter: `name, description` (when Agent should apply it)
- Body: step-by-step instructions; optional `scripts/, references/, assets/`

- Project: `.cursor/skills/<skill>/SKILL.md`
- User-wide: `~/.cursor/skills/`
- Frontmatter: **name**, **description** (when Agent should apply it)
- Body: step-by-step instructions; optional **scripts/**, **references/**, **assets/**
- Agent loads skills at startup; type `/` in chat to invoke by name

- Project: `.cursor/skills/<skill>/SKILL.md`
- User-wide: `~/.cursor/skills/`
- Frontmatter: **name**, **description** (when Agent should apply it)
- Body: step-by-step instructions; optional **scripts/**, **references/**, **assets/**
- Agent loads skills at startup; type `/` in chat to invoke by name
- View loaded skills: **Settings** -> “Rules, Skills, and Subagents”

- Project: `.cursor/skills/<skill>/SKILL.md`
- User-wide: `~/.cursor/skills/`
- Frontmatter: `name, description` (when Agent should apply it)
- Body: step-by-step instructions; optional `scripts/, references/, assets/`
- Agent loads skills at startup; type `/` in chat to invoke by name
- View loaded skills: **Settings** -> “Rules, Skills, and Subagents”
- Make your own with `/create-skill`

- Project: `.cursor/skills/<skill>/SKILL.md`
- User-wide: `~/.cursor/skills/`
- Frontmatter: **name**, **description** (when Agent should apply it)
- Body: step-by-step instructions; optional **scripts/**, **references/**, **assets/**
- Agent loads skills at startup; type `/` in chat to invoke by name
- View loaded skills: **Settings** -> “Rules, Skills, and Subagents”
- Make your own with `/create-skill`
- Example: safe refactor

- Project: `.cursor/agents/<name>.md`; user-wide: `~/.cursor/agents/`

- Project: `.cursor/agents/<name>.md`; user-wide: `~/.cursor/agents/`
- Frontmatter: `name, description`; optional `model, readonly, is_background`

- Project: `.cursor/agents/<name>.md`; user-wide: `~/.cursor/agents/`
- Frontmatter: `name, description`; optional `model, readonly, is_background`
- Body: specialist prompt the parent Agent delegates to

- Project: `.cursor/agents/<name>.md`; user-wide: `~/.cursor/agents/`
- Frontmatter: `name, description`; optional `model, readonly, is_background`
- Body: specialist prompt the parent Agent delegates to
- Create by asking Agent to draft one, or add a file directly with `/create-agent`

- Project: `.cursor/agents/<name>.md`; user-wide: `~/.cursor/agents/`
- Frontmatter: `name, description`; optional `model, readonly, is_background`
- Body: specialist prompt the parent Agent delegates to
- Create by asking Agent to draft one, or add a file directly with `/create-agent`

- Project: `.cursor/agents/<name>.md`; user-wide: `~/.cursor/agents/`
- Frontmatter: `name, description`; optional `model, readonly, is_background`
- Body: specialist prompt the parent Agent delegates to
- Create by asking Agent to draft one, or add a file directly with `/create-agent`

Subagents have their own context, so they can get their own skills and tools, and they can go on long tangents without polluting the parent agent's context.

```
name: verifier
description: "Validates completed work. Use after tasks are marked done
to confirm implementations are functional."
model: fast
---
```

You are a skeptical validator. Your job is to make sure that work claimed to be "done" actually works.

When invoked:

- Identify what was claimed as completed.
- Confirm the implementation exists and works.
- Run the relevant tests or verification steps.
- Look for edge cases that may have been missed.

Be thorough and skeptical. Report:

- What you verified and what passed.
- What was claimed but is incomplete or not working.
- Specific issues that must be fixed.
- Do not take claims at face value. Test everything.

- Step 1: create plan artifact in writing.

- Step 1: create plan artifact in writing.
- Step 2: review against scientific contract.

- Step 1: create plan artifact in writing.
- Step 2: review against scientific contract.
- Step 3: build only approved plan scope.

- Step 1: create plan artifact in writing.
- Step 2: review against scientific contract.
- Step 3: build only approved plan scope.
- Step 4: run fixed verification commands.

- Step 1: create plan artifact in writing.
- Step 2: review against scientific contract.
- Step 3: build only approved plan scope.
- Step 4: run fixed verification commands.
- Repeat until done.

- Human: define research question and non-goals.

- Human: define research question and non-goals.
- Human: decide what would count as a scientific failure.

- Human: define research question and non-goals.
- Human: decide what would count as a scientific failure.
- Planner model: generate detailed implementation plan.

- Human: define research question and non-goals.
- Human: decide what would count as a scientific failure.
- Planner model: generate detailed implementation plan.
- Human: challenge the plan for gaps.

- Human: define research question and non-goals.
- Human: decide what would count as a scientific failure.
- Planner model: generate detailed implementation plan.
- Human: challenge the plan for gaps.
- Builder model: execute bounded implementation tasks.

- Planning/checking: use a stronger reasoning model.

- Planning/checking: use a stronger reasoning model.
- Implementation loops: use a fast coding model.

Match model choice to task phase

- Planning/checking: use a stronger reasoning model.
- Implementation loops: use a fast coding model.
- Escalation/debugging: return to stronger reasoning.

Match model choice to task phase

- Planning/checking: use a stronger reasoning model.
- Implementation loops: use a fast coding model.
- Escalation/debugging: return to stronger reasoning.
- Documentation: use a clear writing model.

- Is the question-to-code mapping explicit?

- Is the question-to-code mapping explicit?
- Are invariants and tests named?

- Is the question-to-code mapping explicit?
- Are invariants and tests named?
- Is data leakage prevented?

- Is the question-to-code mapping explicit?
- Are invariants and tests named?
- Is data leakage prevented?
- Would a human reviewer understand the code without the chat transcript?

- Build one bounded chunk at a time.

- Build one bounded chunk at a time.
- Run the same test command every attempt.

- Build one bounded chunk at a time.
- Run the same test command every attempt.
- Reject unplanned file edits.

- Build one bounded chunk at a time.
- Run the same test command every attempt.
- Reject unplanned file edits.
- Preserve failed outputs for diagnosis.

- Build one bounded chunk at a time.
- Run the same test command every attempt.
- Reject unplanned file edits.
- Preserve failed outputs for diagnosis.
- Commit or checkpoint working states.

- Plan -> check -> build as before.

Advanced scientific workflow: “Prototyper-Teacher”

- Plan -> check -> build as before.
- Worry less about readability of the code – agent equivalent of “just get it working” with minimal effort.

Advanced scientific workflow: “Prototyper-Teacher”

- Plan -> check -> build as before.
- Worry less about readability of the code – agent equivalent of “just get it working” with minimal effort.
- Then ask AI for a reproduction manual (e.g., “write a detailed manual for a beginner to reproduce this codebase from scratch; include exact commands, checks, and file trees”)

Advanced scientific workflow: “Prototyper-Teacher”

- Plan -> check -> build as before.
- Worry less about readability of the code – agent equivalent of “just get it working” with minimal effort.
- Then ask AI for a reproduction manual (e.g., “write a detailed manual for a beginner to reproduce this codebase from scratch; include exact commands, checks, and file trees”)
 - Use a premium, non-coding model with large context window.

Advanced scientific workflow: “Prototyper-Teacher”

- Plan -> check -> build as before.
- Worry less about readability of the code – agent equivalent of “just get it working” with minimal effort.
- Then ask AI for a reproduction manual (e.g., “write a detailed manual for a beginner to reproduce this codebase from scratch; include exact commands, checks, and file trees”)
 - Use a premium, non-coding model with large context window.
 - Manual should enable human-only reimplementation.

Advanced scientific workflow: “Prototyper-Teacher”

- Plan -> check -> build as before.
- Worry less about readability of the code – agent equivalent of “just get it working” with minimal effort.
- Then ask AI for a reproduction manual (e.g., “write a detailed manual for a beginner to reproduce this codebase from scratch; include exact commands, checks, and file trees”)
 - Use a premium, non-coding model with large context window.
 - Manual should enable human-only reimplementation.
- Then re-implement the codebase from scratch using only the manual.

Advanced scientific workflow: “Prototyper-Teacher”

- Plan -> check -> build as before.
- Worry less about readability of the code – agent equivalent of “just get it working” with minimal effort.
- Then ask AI for a reproduction manual (e.g., “write a detailed manual for a beginner to reproduce this codebase from scratch; include exact commands, checks, and file trees”)
 - Use a premium, non-coding model with large context window.
 - Manual should enable human-only reimplementation.
- Then re-implement the codebase from scratch using only the manual.
 - Goal: reduce technical debt from prototype code.

Advanced scientific workflow: “Prototyper-Teacher”

- Plan -> check -> build as before.
- Worry less about readability of the code – agent equivalent of “just get it working” with minimal effort.
- Then ask AI for a reproduction manual (e.g., “write a detailed manual for a beginner to reproduce this codebase from scratch; include exact commands, checks, and file trees”)
 - Use a premium, non-coding model with large context window.
 - Manual should enable human-only reimplementation.
- Then re-implement the codebase from scratch using only the manual.
 - Goal: reduce technical debt from prototype code.
 - Wiping out technical debt will take time and effort.

Advanced scientific workflow: “Prototyper-Teacher”

- Plan -> check -> build as before.
- Worry less about readability of the code – agent equivalent of “just get it working” with minimal effort.
- Then ask AI for a reproduction manual (e.g., “write a detailed manual for a beginner to reproduce this codebase from scratch; include exact commands, checks, and file trees”)
 - Use a premium, non-coding model with large context window.
 - Manual should enable human-only reimplementation.
- Then re-implement the codebase from scratch using only the manual.
 - Goal: reduce technical debt from prototype code.
 - Wiping out technical debt will take time and effort.
 - Allows the human to take accountability for the codebase.

- Environment setup (versions, dependencies).

- Environment setup (versions, dependencies).
- Input/output contract and expected artifacts.

- Environment setup (versions, dependencies).
- Input/output contract and expected artifacts.
- Step-by-step build commands.

- Environment setup (versions, dependencies).
- Input/output contract and expected artifacts.
- Step-by-step build commands.
- Validation checks with expected outcomes.

Projects should in general demonstrate three kinds of control:

Projects should in general demonstrate three kinds of control:

- Scientific control: you define the research question, assumptions, and hypothesis.

Projects should in general demonstrate three kinds of control:

- Scientific control: you define the research question, assumptions, and hypothesis.
- Workflow control: you use Plan mode, bounded prompts, and context tools.

Projects should in general demonstrate three kinds of control:

- Scientific control: you define the research question, assumptions, and hypothesis.
- Workflow control: you use Plan mode, bounded prompts, and context tools.
- Verification control: you test invariants, seeds, edge cases, or sensitivity.

Projects should in general demonstrate three kinds of control:

- Scientific control: you define the research question, assumptions, and hypothesis.
- Workflow control: you use Plan mode, bounded prompts, and context tools.
- Verification control: you test invariants, seeds, edge cases, or sensitivity.

Projects should in general demonstrate three kinds of control:

- Scientific control: you define the research question, assumptions, and hypothesis.
- Workflow control: you use Plan mode, bounded prompts, and context tools.
- Verification control: you test invariants, seeds, edge cases, or sensitivity.

The AI may help build the project but **you own the project**.